

Predicting NBA Injury Risk from Game-Level Workload Data

Alexander Sanchez

1. Problem Definition

Injuries to NBA players have significant financial and competitive consequences, costing the league an estimated \$525 million in 2025–26 (Golliver, 2025). Despite advances in sports analytics, predicting injuries remains challenging because injury risk is complex and multifactorial (Cohan et al., 2021).

This project addresses the following question: Can workload-derived features from game-level NBA data predict whether a player will sustain a non-contact, workload-related IL placement within the next 30 days? Each observation is a player-game row, labeled 1 if an IL placement occurs within 30 days of that game and 0 otherwise. Features include rolling workload metrics, schedule features, and prior injury history.

To address this problem, four classifiers are evaluated: Logistic Regression and Decision Tree as interpretable models, and Random Forest and XGBoost as ensemble models. Given severe class imbalance (~18% positive class), models are evaluated primarily on precision-recall AUC rather than accuracy. The objective is to assess whether interpretable models can match the performance of ensemble models and to quantify the cost of interpretability: how much predictive power, if any, is sacrificed to achieve the transparency required for clinical deployment.

2. Background

2.1 Motivation

NBA injuries are increasingly common despite growing investment in sports medicine and player monitoring. If workload accumulation is a significant driver of this trend, the game-level data already collected by every NBA team contain an early warning signal worth modeling. Bourdon et al. (2017) established that acute spikes in training load relative to a player’s chronic baseline are associated with exponentially increasing injury risk, and Miranda-Comas et al. (2022) demonstrated that even non-sport stressors interact with physical load to elevate risk. Cumulative fatigue, schedule density, and insufficient recovery are modifiable risk factors that can be quantified from observable data.

2.2 Related Work

Cohan et al. (2021) developed METIC, a deep learning model for NBA injury prediction. Their 30-day labeling window and Pro Sports Transactions data pipeline were adopted here, and their finding that contact injuries are the least predictable category informed our decision to exclude contact-driven IL placements from the positive class.

Lu et al. (2022) predicted lower-extremity muscle strain among NBA athletes and found that ensemble models outperformed logistic regression. Their feature-importance results (particularly the dominance of prior injury count and recent region-specific injury flags) directly informed our feature set and our multi-classifier comparison framework.

Our labeling approach yields a higher positive-class rate (~18%) than either prior study, reflecting a flexible definition that retains vague IL placements and soreness/tightness notes as early overuse signals. Known limitations include: IL dates may not reflect true injury onset due to reporting delays, and the 30-day window is a practical rather than clinical choice, poorly suited to acute injuries such as ACL tears. This definition was adopted following Cohan et al. (2021) to maintain methodological consistency and maximize positive-class samples under severe class imbalance, though the resulting label noise likely represents a ceiling on model performance.

2.3 Contribution

While Cohan et al. (2021) used deep learning and Lu et al. (2022) focused on a single injury type, this project explicitly compares interpretable classifiers with ensemble models under a broader, workload-driven injury definition, providing empirical evidence on the trade-off between model transparency and predictive performance in a medical decision-making context where auditable predictions are a practical requirement.

The feature set is constructed entirely from workload-derived game-log features and prior injury history, excluding box score statistics and play-style metrics, thereby isolating the predictive signal attributable to load accumulation and injury recurrence.

3. Dataset

Season	2015–16	2016–17	2017–18	2018–19	2019–20	2020–21	2021–22	2022–23
Injury Rate	0.118	0.148	0.182	0.186	0.179	0.194	0.233	0.209

Table 1: Injury rates by season

3.1 Sources

Game-level player statistics were collected from the NBA Stats API using the `nba_api` Python package, covering eight seasons from 2015–16 through 2022–23 in both regular-season and playoff games. Injury records were sourced from the Kaggle dataset “*NBA Player Injury Stats (1951–2023)*” compiled by Lauton (2023), which fetches data from Pro Sports Transactions. The players’ birthdates were retrieved from the NBA Stats API to compute their ages for each game.

3.2 Quantity

The raw game log data set contains 216,109 player-game entries. After removing irrelevant columns and excluding rows with zero minutes played, the cleaned data set contains 216,088 observations spanning 1,248 unique players across eight seasons, with approximately 500–650 active players per season.

The injury dataset was filtered from 37,668 raw entries to approximately 6,163 workload-relevant injured list (IL) placements, excluding contact injuries, illness, COVID-19 protocols, rest decisions, and return-to-play activations.

3.3 Quality

Each player-game observation was labeled with a binary indicator denoting whether a workload-relevant IL placement occurred within the subsequent 30 days, following the methodology of Cohan et al. (2021). Overlapping injury windows were merged to prevent duplicate labeling. The resulting positive label rate is approximately 18%.

Injury incidence exhibits a clear upward trend over the study period, increasing from 11.8% in 2015–16 to a peak of 23.3% in 2021–22, consistent with the documented rise in NBA injuries league-wide during this interval. Known limitations include label noise due to delayed IL placements, inconsistencies in team reporting, and the arbitrary choice of a 30-day labeling window.

4. Methods

4.1 Data Collection

Game logs for eight seasons (2015–16 through 2022–23) were retrieved from the NBA Stats API using the `nba_api` Python package, with separate pulls for regular-season and playoff games that were combined into a single data set. Injury records were obtained from the Kaggle dataset “*NBA Player Injury Stats (1951–2023)*” and filtered to the study period. The datasets were merged at the player-game level using player identifiers and game dates to create outcome labels and derived features.

4.2 Data Cleaning

Injury filtering. Injury records were filtered to retain only workload-related IL placements. Return-to-play events, non-physical events (e.g., illness, COVID-19, rest), and contact injuries were excluded because they are not predictable from game-level workload features. The resulting dataset focuses on non-contact, load-related injuries.

Player Name standardization. Player names from injury records were matched to NBA API player IDs using fuzzy matching (`rapidfuzz`) at a 90% similarity threshold. Minor manual standardization was applied to resolve common inconsistencies, and unmatched records were excluded.

4.3 Feature Engineering

All features are computed using only information available prior to each game. Rolling and lagged features are constructed to exclude the current game from all lookback windows, thereby preventing data leakage.

Workload features. Player workload was approximated by minutes played. Rolling-minute totals were computed over 7-, 14-, and 28-day windows to capture acute and chronic load, and a 5-game rolling window was also used. Cumulative season minutes were included to represent long-term workload.

Acute:chronic workload ratio. ACWR was defined as the ratio of 7-day to 28-day rolling minutes. A binary indicator was included for values exceeding 1.5, which corresponded to elevated injury risk. Early-season values were excluded until sufficient historical data were available.

Schedule features. Rest days between games, back-to-back indicators, and the number of games played in the prior 7 days were computed to capture scheduling effects.

Player features. Age at each game was included as a continuous variable. Prior injury count was computed as the career-to-date total of IL placements that occurred strictly before each game date, using a time-aware join to prevent leakage.

Recent injury flags. Binary indicators were created for recent injuries (within 56 days) across major body regions, and an aggregate injury history feature was included.

4.4 Modeling

Four classifiers are evaluated: Logistic Regression, Decision Tree, Random Forest, and XGBoost. To address class imbalance (18% positive labels), class weighting and SMOTE oversampling are applied. Models are evaluated using time-based 5-fold cross-validation (`TimeSeriesSplit`), with F1 score, recall, precision, and PR-AUC as the primary metrics.

A key objective is to compare interpretable models (Logistic Regression, Decision Tree) with ensemble models (Random Forest, XGBoost) for practical decision-making.

5. Experiments

5.1 Experimental Setup

The dataset was split chronologically by season to preserve temporal structure: 2015–16 to 2020–21 for training, 2021–22 for validation, and 2022–23 for testing.

Missing values in rolling features were imputed with zero, indicating no prior history, while XGBoost handled missing values natively. Standard scaling was applied only to Logistic Regression.

Class imbalance ($\sim 18\%$ positive) was addressed with class weighting, which performed comparably to or better than SMOTE and was applied to all models. Hyperparameters were tuned using time-based 5-fold cross-validation, with PR-AUC as the primary metric. The following models were evaluated on the test set: Logistic Regression with SMOTE, XGBoost with class weights, Random Forest with class weights, and Decision Tree with class weights.

5.2 Results

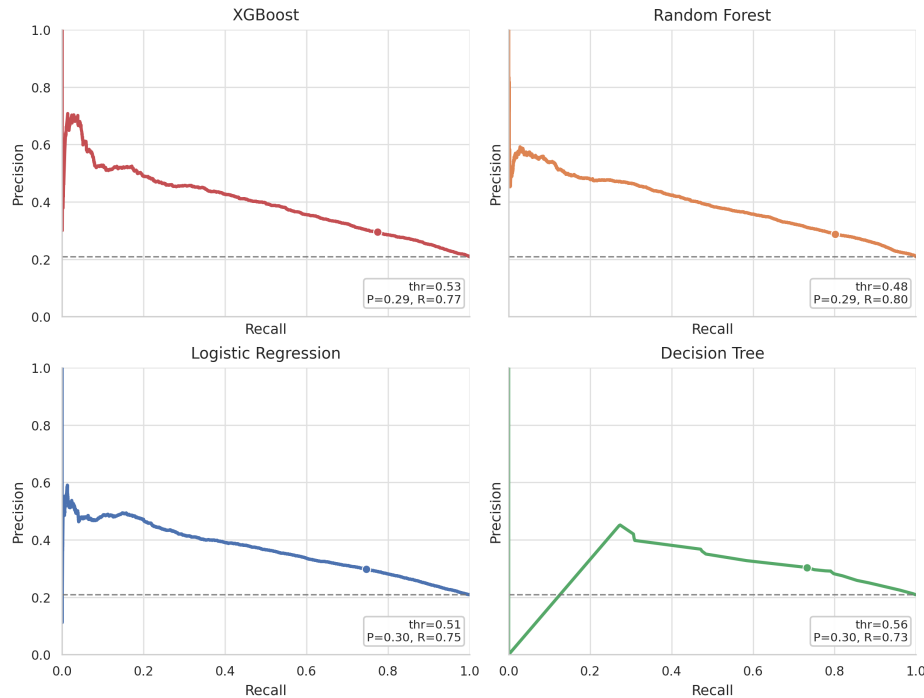


Figure 1: Precision-recall curves on Test Set.

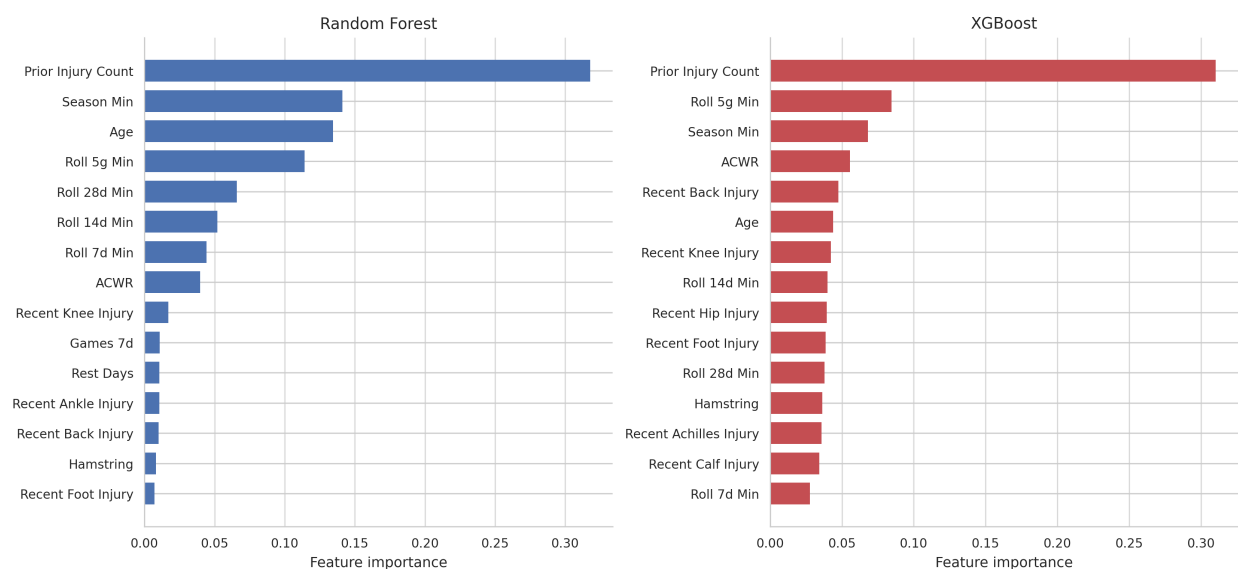


Figure 2: Ensemble Feature Importance on Test Set.

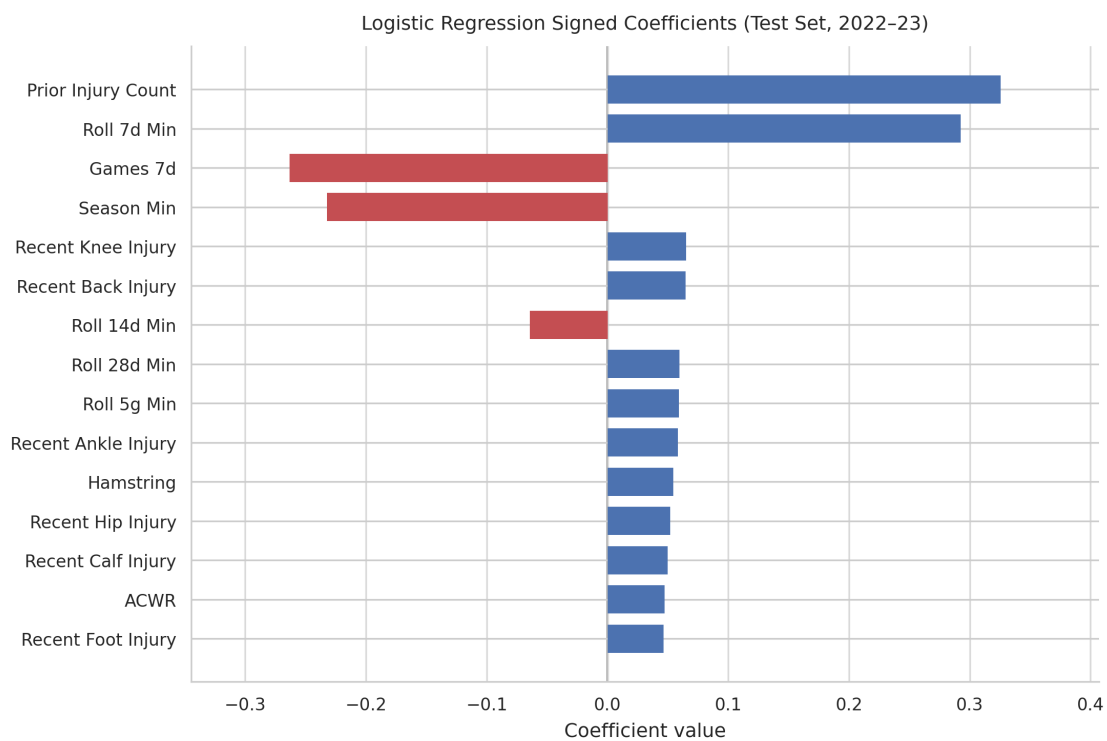


Figure 3: Logistic Regression Signed Coefficients on Test Set.

Model	Val PR-AUC	Test PR-AUC	Δ	Test F1	Test Precision	Test Recall
XGBoost	0.349	0.396	+0.047	0.427	0.295	0.774
Random Forest	0.338	0.391	+0.052	0.423	0.287	0.802
Logistic Regression	0.360	0.366	+0.005	0.426	0.298	0.747
Decision Tree	0.332	0.347	+0.014	0.429	0.303	0.733

Table 2: Model performance comparison

5.3 Analysis

Although Logistic Regression achieved the highest validation PR-AUC score, ensemble models outperformed in the test set, with XGBoost reaching a PR-AUC of 0.396. This may reflect ensemble models’ ability to capture local interactions among correlated workload features that a linear model cannot represent with a single global set of weights.

All models converge to a similar operating point ($F1 \approx 0.42$ – 0.43 , $\text{recall} \approx 0.73$ – 0.80 , $\text{precision} \approx 0.29$ – 0.30), indicating strong recall at the cost of increased false positives. This tradeoff is appropriate for risk screening but limits direct intervention.

Feature importance was consistent across models: prior injury history was the strongest predictor, followed by workload features. Workload signals were distributed across multiple correlated features, whereas injury history was captured by a single dominant variable.

Overall performance plateaued around 0.40 PR-AUC, reflecting the limitations of using game-level workload data without biometric or tracking inputs.

6. Observation and Conclusion

Ensemble models outperformed interpretable models on the test set, with XGBoost achieving the highest PR-AUC (0.396). Although workload features were informative, prior injury history was the strongest predictor across all models, indicating that injury recurrence is a dominant signal in this dataset.

Several limitations affect these results. Model selection based on a single validation season proved sensitive to shifts in injury-prevalence distributions across seasons. All features were derived from publicly available game logs, limiting predictive performance. Label noise from delayed IL reporting and the fixed 30-day window further introduced uncertainty into the outcome variable.

Future work should explore more robust time-series validation and incorporate richer data sources to improve predictive accuracy. Despite these limitations, the model demonstrates that meaningful injury risk signals can be extracted from game-level data, offering a practical tool for early risk screening.

References

- [1] Bourdon, P. C., Cardinale, M., Murray, A., Gastin, P., Kellmann, M., Varley, M. C., Gabbett, T. J., Coutts, A. J., Burgess, D. J., Gregson, W., & Cable, N. T. (2017). Monitoring athlete training loads: Consensus statement. *International Journal of Sports Physiology and Performance*, 12(Suppl. 2), S2-161–S2-170. <https://doi.org/10.1123/IJSP.2017-0208>
- [2] Cohan, A., Schuster, J., & Fernandez, J. (2021). A deep learning approach to injury forecasting in NBA basketball. *Journal of Sports Analytics*, 7(3), 277–289. <https://doi.org/10.3233/JSA-200529>
- [3] Golliver, B. (2025, December 10). The NBA's \$525 million injury epidemic. *The Washington Post*. <https://www.washingtonpost.com/sports/2025/12/10/nba-injuries/>
- [4] Lauton, L. (2023). *NBA player injury stats 1951–2023* [Dataset]. Kaggle. <https://www.kaggle.com/datasets/loganlauton/nba-injury-stats-1951-2023>
- [5] Lu, Y., Pareek, A., Lavoie-Gagne, O. Z., Forlenza, E. M., Patel, B. H., Reinholz, A. K., Forsythe, B., & Camp, C. L. (2022). Machine learning for predicting lower extremity muscle strain in National Basketball Association athletes. *Orthopaedic Journal of Sports Medicine*, 10(7), 23259671221111742. <https://doi.org/10.1177/23259671221111742>
- [6] Miranda-Comas, G., Zaman, N., Ramin, J., & Gluck, A. (2022). The role of workload management in injury prevention, athletic, and academic performance in the student-athlete. *Current Sports Medicine Reports*, 21(6), 186–191. <https://doi.org/10.1249/JSR.0000000000000964>
- [7] Sanchez, A. (2026). *NBA injury risk prediction* [Software]. GitHub. <https://github.com/tumbrelpeanuts/nba-injury-risk/tree/main>
- [8] Swar. (2020). *nba_api: An API client package to access the APIs for NBA.com* [Software]. GitHub. https://github.com/swar/nba_api